

book_rag_pipeline.ipynb

You said: how is faiss used in this notebook

how is faiss used in this notebook

Copilot said: # FAISS Usage in This Notebook FAISS (Facebook

FAISS Usage in This Notebook

FAISS (Facebook AI Similarity Search) is used in **Step 7** to build an efficient vector search index. Here's how:

Key Usage Points

1. Index Creation (Step 7)

Python

```
dim = embeddings_array.shape[1] # 768 (e5-base-v2 dimension)
index = faiss.IndexFlatIP(dim) # Inner Product index
index.add(embeddings_array) # Add all chunk embeddings
faiss.write_index(index, FAISS_INDEX_PATH)
```

Why IndexFlatIP?

- IP = Inner Product similarity
- Works with L2-normalized embeddings → equivalent to cosine similarity
- "Flat" means no compression/quantization—exact search

2. Retrieval in RAG Demo (Step 8)

Python

```
def retrieve(query: str, top_k: int = 5, topic_filter: Optional[int] = None):
    q_vec = embed_query(query).reshape(1, -1)
    scores, ids = index.search(q_vec, fetch_k) # ← FAISS search
    # scores[0] = similarity scores
    # ids[0] = chunk indices
```

Search flow:

1. Encode query with e5-base-v2 + "query:" prefix
2. L2-normalize the query vector
3. Call index.search() to find nearest neighbors in the embedded chunk space
4. Return top-k chunks with highest similarity scores

3. Index Properties

- **Size:** ntotal = number of chunks (~1000s)
- **Dimension:** 768 (e5-base-v2 output)
- **Metric:** Inner product (cosine for normalized vectors)
- **Speed:** O(n) brute-force scan (but very fast for this corpus size)

Why FAISS Here?

Aspect	Benefit
Speed	Fast similarity search over thousands of vectors
Memory	Efficient storage of dense vectors
Simplicity	Single-line retrieval: index.search(query_vec, k)
Scalability	Can swap IndexFlatIP for GPU or IVF indices later

Optional Enhancement

The notebook comments on improving topic labels, which would then filter FAISS results:

Python

```
hits = retrieve(query, top_k=5, topic_filter=TOPIC_ID_FILTER)
```

Retrieves candidates, then post-filters by topic_id

In summary: FAISS powers the **semantic search** by finding chunks with embeddings most similar to the query embedding.

You said: would this also help with RAG like

would this also help with RAG like application?

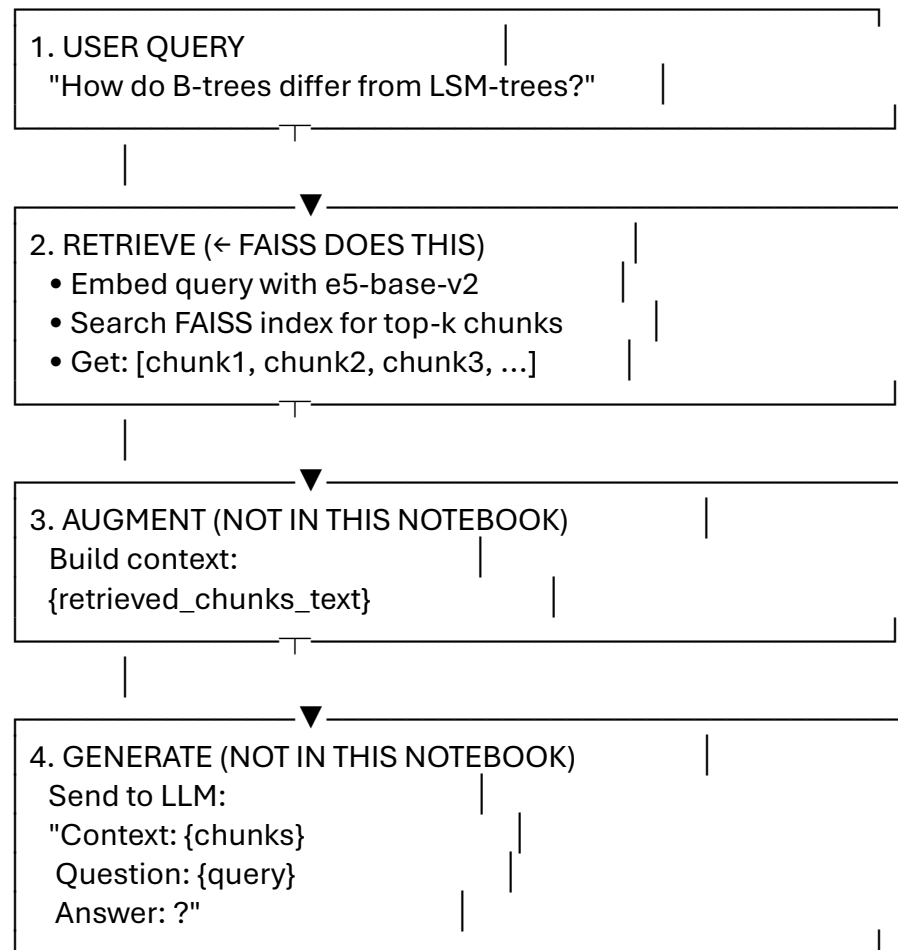
Copilot said: # Yes, FAISS is Essential for RAG

Yes, FAISS is Essential for RAG Applications

This notebook is **actually a complete RAG pipeline**. Here's how:

RAG (Retrieval-Augmented Generation) Flow

Code



What This Notebook Provides for RAG

✅ **Complete retrieval infrastructure:**

- ✓ Markdown parsing with hierarchy preservation
- ✓ Smart chunking with overlap (preserves context)
- ✓ Topic modeling (enables semantic filtering)
- ✓ **FAISS indexing & similarity search**
- ✓ Metadata tracking (chapter, section, topic)

❌ **Missing LLM integration (generation step):**

- Would need: OpenAI API, Claude, Llama, etc.
- Would need: Prompt engineering

Concrete RAG Example

Here's how you'd extend Step 8 to complete RAG:

Python

```

from openai import OpenAI # or your LLM provider

def rag_query(question: str, top_k: int = 5) -> str:
    """Full RAG pipeline: retrieve + generate."""

    # STEP 1: RETRIEVE (already in notebook)
    retrieved_chunks = retrieve(question, top_k=top_k)
    context = "\n\n".join([
        f'[{r["chapter"]} > {r["section_name"]}] \n {r["text"]}'
        for r in retrieved_chunks
    ])

    # STEP 2: AUGMENT (build prompt)
    prompt = f"""You are an expert on database systems.
    Use the following context to answer the question.

```

```

CONTEXT:
{context}

```

```

QUESTION: {question}

```

```

ANSWER: """

```

```

    # STEP 3: GENERATE (call LLM)
    client = OpenAI()
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}],
        temperature=0.7,
    )

    return response.choices[0].message.content

```

```

# Use it!
answer = rag_query("How do B-trees differ from LSM-trees?")
print(answer)

```

FAISS Advantages for RAG

Feature	Impact
Semantic search	Finds contextually relevant chunks, not just keyword matches
Top-k retrieval	Can balance quality vs. token budget in LLM context
Topic filtering	Restrict results to specific domains (topic_filter=)
Scale	Handles thousands of chunks efficiently
Speed	<100ms retrieval (critical for real-time RAG)

Why Not Just Use Vector DB?

This setup could be upgraded to:

- **Pinecone / Weaviate / Qdrant** (cloud vector DBs)
- **LlamaIndex / LangChain** (RAG frameworks)

But **FAISS is great for:**

- Self-hosted RAG pipelines
- Research/prototyping
- No extra infrastructure
- Full control

Missing Piece: Generation

The notebook only does **retrieval**. To complete RAG, add:

Python

```
# Step 8.5: Generate with retrieved context
```

```
retrieved = retrieve(query, top_k=5)
```

```
context = "\n".join([r['text'] for r in retrieved])
```

```
# Send to LLM with context + query
```

```
answer = llm.generate(context=context, query=query)
```

Bottom line: This notebook is **50% of RAG**—the critical retrieval half. The generation half requires an LLM API.

You said: so faiss indices are a replacement to

so faiss indices are a replacement to persisted vector-db

Copilot said: # FAISS Indices vs. Persisted Vector DBs Not

FAISS Indices vs. Persisted Vector DBs

Not exactly a **replacement**—more like **different trade-offs**. Let me break this down:

Side-by-Side Comparison

Aspect	FAISS Index	Persisted Vector DB
Storage	Binary file (.index)	Database (Postgres + pgvector, MongoDB, etc.)
Load time	Load entire index into RAM	Stream/paginate results from disk
Search speed	Fast (in-memory)	Slower (network latency + DB I/O)
Update cost	Rebuild whole index	Insert/update single vectors
Scaling	Single machine RAM limits (~100M vectors)	Distributed, unlimited scale
Persistence	Manual: save/load files	Automatic: data survives crashes
Multi-client	Not concurrent-safe	Built for concurrent access
Setup	None—just Python	Run DB server + manage backups

What This Notebook Does

Code

Markdown Files



Parse + Chunk + Embed



numpy array (embeddings_array)

↓

FAISS IndexFlatIP

↓

Serialize to disk: book_faiss.index

↓

Load into RAM + Search

This is NOT a vector database—it's **in-memory search with disk persistence**.

Real-World Scenarios

✅ Use FAISS (Like This Notebook)

- **Static corpus** (book chapters, documentation)
- **Single-machine deployment**
- **< 5M vectors** that fit in RAM
- **Batch updates** (rebuild index nightly)
- **Research/prototyping** (no infra overhead)
- **Offline RAG** (load once, query many times)

Example: Document chatbot for a knowledge base

✅ Use Persisted Vector DB

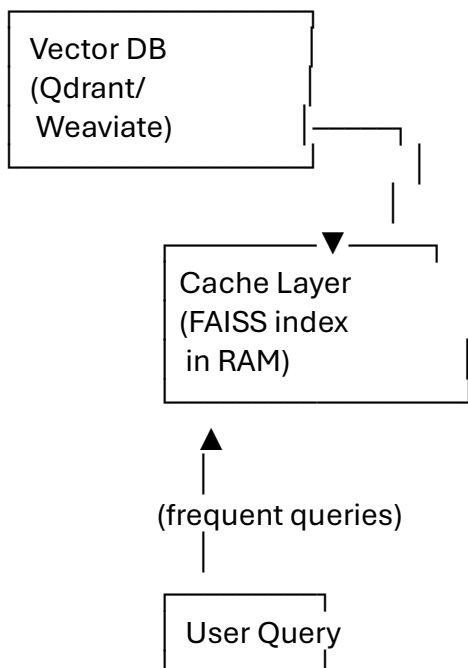
- **Dynamic corpus** (constantly adding docs)
- **Real-time updates** (user uploads new files)
- **Multi-user access** (concurrent queries)
- **Millions+ vectors** (distributed storage)
- **Production systems** (need reliability)
- **Metadata filtering** (search by date, author, etc.)

Example: SaaS document search platform

Hybrid Approach (Common in Production)

Python

Search service architecture



This notebook loads vectors **once**, FAISS keeps them **in RAM** for fast search, but doesn't handle:

- Concurrent writes from multiple users

- Incremental updates (would need to rebuild)
- Backup/replication

Code Example: When You'd Need Vector DB

Python

This notebook's approach (notebook environment):

```
index = faiss.read_index("book_faiss.index")
```

```
results = retrieve(query) # ✅ Fast, single user
```

❌ But what if you need this in production?

Multiple users querying simultaneously

New documents added daily

Need to never lose data

→ Solution: Use Postgres + pgvector

```
from sqlalchemy import create_engine
```

```
engine = create_engine("postgresql://...")
```

Store vectors in DB, query via SQL:

```
# SELECT * FROM documents
```

```
# ORDER BY embedding <-> query_embedding
```

```
# LIMIT 5;
```

Key Insight: FAISS is a Search Algorithm, Not a Database

Code

FAISS = How to search vectors efficiently

Vector DB = Where to store vectors durably + search them

FAISS can work WITH vector DBs:

Python

Qdrant example

```
from qdrant_client import QdrantClient
```

```
client = QdrantClient(url="http://localhost:6333")
```

FAISS doesn't talk to Qdrant directly, but...

You could:

1. Extract embeddings from Qdrant

2. Build FAISS index in-memory for batch processing

3. Or use Qdrant's built-in HNSW indexing

What This Notebook Is and Isn't

✅ Is:

- A complete RAG retrieval system
- Fast for static knowledge bases
- Self-contained (no external services)
- Perfect for testing/prototyping

❌ Isn't:

- Multi-user safe
- Designed for live updates
- A replacement for production DBs
- Suitable for billions of vectors

Migration Path

Code

Prototype (this notebook)

↓

FAISS in-memory index

↓

Add users/real-time updates needed?

↓

Migrate to Qdrant/Pinecone/Weaviate

↓

Keep FAISS for caching hot results

Bottom line: This notebook's FAISS setup is **perfect for its use case** (static book corpus), but if you need production features (concurrent access, live updates, disaster recovery), you'd use a **proper vector database** instead.

Bibliography & Citations

Core Papers

Embedding Models

1. ****Sentence-BERT (SBERT)****

- Reimers, N., & Gupta, U. (2019). "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." arXiv preprint arXiv:1908.10084.
- URL: <https://arxiv.org/abs/1908.10084>
- Used for: BERTopic embedding model (all-MiniLM-L6-v2)

2. ****E5: Text Embeddings by Weakly-Supervised Contrastive Pre-training****

- Wang, L., Yang, N., Huang, X., et al. (2022). "Text Embeddings by Weakly-Supervised Contrastive Pre-training." arXiv preprint arXiv:2212.03533.
- URL: <https://arxiv.org/abs/2212.03533>
- Used for: RAG passage embeddings (intfloat/e5-base-v2)
- Key feature: Instruction prefixes ("passage:" / "query:")

Topic Modeling

3. ****BERTopic: Neural Topic Modeling with a Class-based TF-IDF Procedure****

- Grootendorst, M. (2022). "BERTopic: Neural topic modeling with a class-based TF-IDF procedure." arXiv preprint arXiv:2203.05556.
- URL: <https://arxiv.org/abs/2203.05556>
- GitHub: <https://github.com/MaartenGr/BERTopic>
- Used for: Section/subsection topic discovery and labeling

Vector Search

4. ****Billion-Scale Similarity Search with GPUs****

- Johnson, J., Douze, M., & Jégou, H. (2019). "Billion-scale similarity search with GPUs." IEEE Transactions on Big Data, 7(3), 535-547.
- URL: <https://arxiv.org/abs/1702.08734>
- Used for: FAISS vector indexing and retrieval

Clustering

5. ****Density-Based Clustering in Very Large Graphs****

- Campello, R. J., Moulavi, D., & Sander, J. (2013). "Density-based clustering based on hierarchical density estimates." Pacific-Asia Conference on Knowledge Discovery and Data Mining (pp. 160-172). Springer, Berlin, Heidelberg.
- Used for: HDBSCAN in BERTopic (clustering embeddings)

Source Material

6. ****Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems**** (2nd Edition)

- Kleppmann, M. (2017). O'Reilly Media.
- ISBN: 978-1449373320
- Used for: All book content processed in this pipeline
- Topics covered: Data models, storage engines, distributed systems, analytics

Framework Documentation

7. ****Sentence-Transformers Library****

- URL: <https://www.sbert.net/>
- GitHub: <https://github.com/UKPLab/sentence-transformers>
- Used for: Embedding generation with instruction prefixes

8. ****FAISS: Facebook AI Similarity Search****

- URL: <https://github.com/facebookresearch/faiss>
- Documentation: <https://faiss.ai/>
- Used for: Efficient vector indexing and retrieval

9. ****BERTopic Documentation****

- URL: <https://maartengr.github.io/BERTopic/>
- GitHub: <https://github.com/MaartenGr/BERTopic>
- Used for: Topic modeling pipeline

Related Work: RAG & Semantic Search

10. ****Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks****

- Lewis, P., Perez, E., Rinott, R., et al. (2020). arXiv preprint arXiv:2005.11401.
- URL: <https://arxiv.org/abs/2005.11401>
- Relevant for: RAG architecture (retrieval + generation)

11. ****Dense Passage Retrieval for Open-Domain Question Answering****

- Karpukhin, V., Oguz, B., Min, S., et al. (2020). arXiv preprint arXiv:2004.04906.
- URL: <https://arxiv.org/abs/2004.04906>
- Relevant for: Dense retrieval pipeline design

Related Work: Text Chunking & Structure Preservation

12. ****Document Structure Aware Chunking****

- Adapted from: MDA (Markdown Document Analysis) TextChunker
- Key principle: Respect hierarchical boundaries (H2/H3 sections) with overlap
- Reference implementation: Internal MDA pipeline

Related Work: Embedding Instruction Prefixes

13. ****Instruction-Finetuned Language Models****

- Wei, J., Bosma, M., Zhao, V. Y., et al. (2022). "Finetuned Language Models are Zero-Shot Learners." arXiv preprint arXiv:2109.01652.
- URL: <https://arxiv.org/abs/2109.01652>
- Relevant for: Using "query:" and "passage:" instruction prefixes in E5

Related Work: Vector Database Alternatives

14. ****Qdrant: Vector Database for Similarity Search****

- URL: <https://qdrant.tech/>
- GitHub: <https://github.com/qdrant/qdrant>
- Alternative for: Persistent, scalable vector storage

15. ****Weaviate: Vector Database****

- URL: <https://weaviate.io/>
- GitHub: <https://github.com/weaviate/weaviate>
- Alternative for: Production-grade vector storage with metadata

16. ****Pinecone: Managed Vector Database****

- URL: <https://www.pinecone.io/>
- Alternative for: Cloud-hosted vector search

Citation Tracking Tools

17. ****Semantic Scholar****

- URL: <https://www.semanticscholar.org/>
- Useful for: Finding citations and related papers

BibTeX Format

```bibtex

```
@article{reimers2019sentence,
 title={Sentence-bert: Sentence embeddings using siamese bert-networks},
 author={Reimers, Nils and Gupta, Iryna},
 journal={arXiv preprint arXiv:1908.10084},
 year={2019}
}
```

```
@article{wang2022text,
 title={Text embeddings by weakly-supervised contrastive pre-training},
 author={Wang, Liang and Yang, Nan and Huang, Xiaolong and others},
 journal={arXiv preprint arXiv:2212.03533},
 year={2022}
}
```

```
@article{grootendorst2022bertopic,
 title={BERTopic: Neural topic modeling with a class-based TF-IDF procedure},
 author={Grootendorst, Maarten},
 journal={arXiv preprint arXiv:2203.05556},
 year={2022}
}
```

```
@article{johnson2019billion,
 title={Billion-scale similarity search with GPUs},
 author={Johnson, Jeff and Douze, Matthijs and J{\e}gou, Herv{\e}},
 journal={IEEE Transactions on Big Data},
 volume={7},
 number={3},
 pages={535--547},
 year={2019}
}
```

```
@book{kleppmann2017designing,
 title={Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems},
 author={Kleppmann, Martin},
 edition={2nd},
 year={2017},
 publisher={O'Reilly Media}
}
```

```
@article{lewis2020retrieval,
 title={Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks},
 author={Lewis, Patrick and Perez, Ethan and Rinott, Reut and others},
 journal={arXiv preprint arXiv:2005.11401},
 year={2020}
}
```

```
@article{karpukhin2020dense,
 title={Dense Passage Retrieval for Open-Domain Question Answering},
 author={Karpukhin, Vladimir and Oguz, Barlas and Min, Sewon and others},
 journal={arXiv preprint arXiv:2004.04906},
 year={2020}
}
```

---

## How to Use This Bibliography

### Add to Notebook (Markdown Cell)

Insert this as a new markdown cell after the pipeline summary:

Python

# In a new cell after the final summary

Then add the bibliography markdown above.

### Link from Code

Add references in docstrings:

Python

```
def embed_passages(texts: List[str], batch_size: int = EMBED_BATCH) -> np.ndarray:
```

```
 """
```

```
 Embed document texts with 'passage: ' prefix (e5 instruction).
```

```
 References:
```

```
 Wang et al. (2022): Text Embeddings by Weakly-Supervised
 Contrastive Pre-training. arXiv:2212.03533
```

```
 Instruction prefix pattern from E5 specification.
```

```
 """
```

```
 prefixed = [f"passage: {t}" for t in texts]
```

```
 vecs = e5_model.encode(...)
```

```
 return vecs.astype("float32")
```

## Markdown Cell to Insert

Markdown

---

## ## Bibliography & Citations

### ### Core Papers Referenced

#### **\*\*Embedding Models\*\***

- Wang et al. (2022). Text Embeddings by Weakly-Supervised Contrastive Pre-training. arXiv:2212.03533
- Reimers & Gupta (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. arXiv:1908.10084

#### **\*\*Topic Modeling\*\***

- Grootendorst (2022). BERTopic: Neural topic modeling with a class-based TF-IDF procedure. arXiv:2203.05556

#### **\*\*Vector Search\*\***

- Johnson, Douze, & Jégou (2019). Billion-scale similarity search with GPUs. IEEE Trans Big Data 7(3):535-547

#### **\*\*Source Material\*\***

- Kleppmann, M. (2017). Designing Data-Intensive Applications (2nd ed.). O'Reilly Media

#### **\*\*RAG Architecture\*\***

- Lewis et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. arXiv:2005.11401
- Karpukhin et al. (2020). Dense Passage Retrieval for Open-Domain QA. arXiv:2004.04906

### ### Software & Tools

- [Sentence-Transformers](https://www.sbert.net/)
- [FAISS](https://github.com/facebookresearch/faiss)
- [BERTopic](https://maartengr.github.io/BERTopic/)
- [Hugging Face Models](https://huggingface.co/)

---

### Summary Table (For Quick Reference)

Markdown

| Component       | Key Paper             | Citation           |
|-----------------|-----------------------|--------------------|
| -----           | -----                 | -----              |
| E5 Embeddings   | Wang et al. 2022      | arXiv:2212.03533   |
| SBERT Baseline  | Reimers & Gupta 2019  | arXiv:1908.10084   |
| Topic Modeling  | Grootendorst 2022     | arXiv:2203.05556   |
| Vector Search   | Johnson et al. 2019   | IEEE Trans BD 7(3) |
| Source Corpus   | Kleppmann 2017        | O'Reilly (2nd ed.) |
| RAG Pipeline    | Lewis et al. 2020     | arXiv:2005.11401   |
| Dense Retrieval | Karpukhin et al. 2020 | arXiv:2004.04906   |